
ALEKSANDR RIBINSKI

Python Backend developer, 6+ years experience

+7 (926) 660-3633

Telegram: t.me/aleksandr_developer_backend
UTC+4 (Tbilisi Georgia)

aleksandr.riabinski@gmail.com

[LinkedIn](#)

[Github](#)

ABOUT

Python Backend Developer with 6+ years of commercial experience building and supporting production backend systems across fintech, edtech, and industrial domains. Experienced in system architecture and end-to-end backend delivery: from business requirements and API contracts to data storage, integrations, observability, and production support. Strongest at building backend systems that process business-critical data, automate internal workflows, integrate with external services, and stay reliable under production load.

SKILLS

Main: Python, FastAPI, Django, PostgreSQL, Redis, Docker, Linux, Git, SQLAlchemy, Pytest, REST API, asyncio, RabbitMQ, Celery, Kubernetes, AWS, GraphQL, gRPC, MySQL, MongoDB, Alembic, Nginx, GitLab, BitBucket, GitLab CI, Prometheus, Grafana, Poetry, Elasticsearch, Kafka, Flask, DRF, WebSockets, Azure, Terraform, Ansible, Jenkins, SonarQube, Docker, Podman, Docker Compose, Cassandra, Selenium, Postman, Insomnia, Bash, Pandas, NumPy, langchain

Other: HTML, CSS, JavaScript, go, c++

EXPERIENCE

SBERBANK: 10/2023 - Present, 2+ YEAR

Product: Internal services of the bank

Achievements:

- Designed and built an autonomous AI incident-investigation agent for a 30+ microservice banking product, reducing production issue investigation from hours or occasionally days to minutes by analyzing centralized logs, retrieving service/code context, identifying likely root causes, and sending messenger alerts with incident summaries and recommended fixes.
- Implemented and evangelized Spec-Driven Development using OpenSpec, establishing specification-first pipelines across all team services so user stories, bugs, and releases were consistently designed, validated, and delivered from explicit technical specs; reduced development time and accelerated the path from development to production by improving requirement clarity and delivery predictability.
- Implemented and evangelized end-to-end AI-agent tracing with Arize Phoenix across the team, instrumenting 10+ production services with structured spans for agent runs, LLM calls, tool execution, retrieval steps, external service calls, retries, and errors; reduced debugging time, improved visibility into failed agent decisions and latency bottlenecks, and helped satisfy bank reliability requirements for production AI services.
- Designed and built an internal LangChain wrapper library for bank LLM integrations, standardizing retry policies, circuit breakers, timeout handling, structured-output validation with JSON/schema/Pydantic contracts, automatic response repair, fallback behavior, prompt/version metadata, sensitive-data masking, and Redis-backed certificate counters for distributed concurrency control; reduced duplicated integration code and improved reliability, security, and contract consistency for services consuming bank LLM models.
- Implemented production RAG pipelines for analytical reports and internal bank news, consuming content from the bank news and analytics team through Kafka, normalizing and chunking documents, generating embeddings, and indexing knowledge sources for semantic search; improved answer relevance and reduced manual search time by enabling AI services to retrieve up-to-date business context for internal users.
- Implemented AI-powered code review automation in the internal bank repository platform, triggering an internal review agent for every pull request to analyze diffs, detect risky changes, validate team conventions, and publish actionable feedback directly in the PR; reduced routine review effort and improved early defect detection before human review.
- Created and promoted a standardized service bootstrap template for new microservices across the team and adjacent teams, embedding production-ready defaults for logging, metrics, clean architecture structure, required internal libraries, OpenSpec specifications, uv-based dependency management, Docker setup, and base configuration; accelerated service creation and improved architectural consistency across teams.

Technologies: Python, Go, FastAPI, gRPC, GraphQL, Grafana, Helm, LangChain, LangGraph, PostgreSQL, BitBucket, Jenkins, SonarQube, Docker, Kubernetes, OpenShift, MongoDB, GreenPlum, Redis, Kafka, smtp, OpenSearch, clickhouse

VK: 12/2020 - 10/2023, 2 YEARS 8 MONTH

Product: Tetrika

An online platform for preparing for the Unified State Exam, and improving school performance. More than 110,000 students study on the platform. The project's revenue for 2024 is 46 million dollars

Achievements:

- Developed backend features for the Lesson Constructor, enabling tutors to assemble interactive lesson programs from a shared library of methodist-prepared materials; reduced lesson preparation time, improved reuse of educational content, and helped standardize lesson quality across recurring classes.
- Created backend functionality for mini-group lessons, extending lesson, schedule, and billing logic from one-to-one tutoring to group-based classes; enabled a lower-cost learning format, reduced duplicate implementation across lesson flows, and kept teacher schedules, student enrollment, and lesson accounting consistent.
- Developed backend APIs for the student and parent personal account redesign, supporting schedule, lesson materials, homework, lesson balance, purchase history, and payment status data; improved transparency of the learning process, reduced support requests about lessons and payments, and made key learning data easier to access.
- Implemented backend functionality for purchasing lesson packages, connecting package selection, payment processing, installment payments, lesson balance updates, and purchase history into a consistent billing flow; improved payment transparency for students and parents and increased reliability of paid lesson accounting.

- Developed backend functionality for lesson rescheduling and cancellation workflows, synchronizing schedule changes with lesson statuses, notifications, student balances, and teacher availability; reduced inconsistent lesson states, prevented incorrect balance write-offs, and improved reliability of changed or canceled lessons.

Python, Tornado, FastAPI, PostgreSQL, Docker, Kubernetes, Redis, SQLAlchemy, gRPC, Kafka, aiohttp, pyjwt, bcrypt, pandas, numpy, Cassandra, RabbitMQ, Celery, MongoDB, GraphQL, WebSockets, Aioboto3, S3, httpx, Twilio, aiomysql

ROSTEC: 11/2019 - 12/2020, 1 YEARS 1 MONTH

Product: Digital manufacturing management platform

The platform helped industrial enterprises digitize production workflows: technologists created manufacturing process cards, planners built production plans, operators executed work on equipment, machines sent status events, and managers tracked production history through dashboards and reports. The system connected technological documentation, reference data, equipment monitoring, and digital production passports into a unified production management workflow.

Achievements:

- Developed early backend modules for digital production passports, enabling engineers to store structured manufacturing data, production stages, materials, equipment references, and operation history in a unified system; improved traceability of production records and reduced reliance on fragmented spreadsheets.
- Developed backend APIs for managing manufacturing process cards, allowing technologists to define ordered production operations, required materials, equipment, operation duration, and document versions; improved traceability and reduced errors caused by outdated or duplicated process documents.
- Built backend APIs for equipment monitoring dashboards, processing machine status events and exposing current state, downtime, workload, and error metrics; improved visibility into workshop operations and helped operators detect equipment problems faster.
- Implemented asynchronous processing for production planning and reporting tasks with Celery and RabbitMQ, moving long-running workload calculations and report generation out of HTTP requests; reduced API timeouts and improved reliability of operational data updates.

Technologies: Python, Django, PostgreSQL, Redis, Celery, RabbitMQ, Linux, Bash, Marshmallow, Docker, requests, pandas, matplotlib, numpy, gRPC

EDUCATION

Moscow Aviation Institute, Moscow (Russia) – BACHELOR, 2018 - 2022

Information and computer science